

Daily Routine & E-Book Tracker

Project Requirements Document • v2.0

A cross-platform React Native app combining habit tracking, Neumorphic 3D UI, immersive scroll-triggered motion, and a personal + public e-book library — all governed by a powerful admin console.

Version	Date	Status	Platforms
2.0.0	May 2026	Planning → Build	Android • iOS (React Native)

Highlights of v2.0

- Full migration from PWA to native React Native (Android & iOS).
- Neumorphic / 3D UI system — every button, card and toggle has depth.
- Scroll-triggered animations powered by Reanimated 3 + Moti (React Native equivalent of Framer Motion).
- Built-in **E-Book Reader** with online streaming and offline download for reading without internet.
- Users can **upload their own books** (PDF / EPUB) and optionally make them public to a community library.
- Dedicated **Admin Panel** (web + mobile) for moderation, feature flags, user management and analytics.
- Reliable **push-notification reminders** for every scheduled routine.

Table of Contents

#	Section
1	Executive Summary
2	Product Definition
3	Target Users & Personas
4	Feature Catalogue (with new modules)
5	Technology Stack
6	3D / Neumorphic Design System
7	Motion & Scroll-Triggered Animations
8	E-Book Reader — Online & Offline
9	User Uploads & Public Library
10	Admin Panel & Role Model
11	Push-Notification Reminder System
12	Database Schema
13	REST + Realtime API Endpoints
14	Frontend Architecture (React Native)
15	Backend Architecture
16	Security, Auth & Privacy
17	Stats & Results Engine
18	DevOps & Deployment Pipeline
19	Development Phases & Timeline
20	Non-Functional Requirements
21	Testing Strategy
22	Risks & Mitigations
23	Master AI Build Prompt

1. Executive Summary

The Daily Routine & E-Book Tracker is a native mobile app that helps users build consistent daily habits while giving them a beautiful place to read. It merges a productivity tracker with a personal library — both wrapped in a tactile, Neumorphic 3D interface and animated with scroll-triggered motion. Reminders nudge users at the right time; the admin console gives operators full control over content moderation and feature rollouts.

Built on React Native (single codebase for Android & iOS), the app uses Reanimated 3 and Moti to deliver Framer-Motion-style animations natively, Firebase Cloud Messaging for cross-platform push, and a hardened Node.js + PostgreSQL backend with role-based access control.

Primary outcomes

- Users complete more of their daily routines (target: +35% completion rate).
- Users spend more relaxed, screen-positive time reading (target: 4 reading sessions/week per active user).
- Operators can moderate, scale and monetise without engineering involvement.

2. Product Definition

Vision Statement

Build a delightful, tactile companion that turns daily routines into rituals — and gives users a calm, private space to read books that support those routines.

Problem Statement

- Routine trackers feel flat and forgettable — there is no tactile feedback or sense of depth.
- Most productivity apps don't surface the knowledge (books, guides) that actually helps people change behaviour.
- Self-uploaded reading material is scattered across cloud drives with no offline-first reader.
- There is no governance layer for safe sharing of user-contributed content.

Differentiators

- **Neumorphic 3D UI** — buttons, cards and toggles look pressable; shadows, highlights and elevation change on touch.
- **Cinema-grade motion** — every scroll, swipe and section reveal is choreographed with Reanimated worklets.
- **Library + Tracker in one** — recommended reading is tied to the user's routines (e.g. 'Atomic Habits' for a morning streak).
- **Bring-your-own-book** — users upload PDFs/EPUBs and choose private or public visibility.
- **Operator-first admin panel** — full moderation, analytics, feature flags and broadcast notifications.

3. Target Users & Personas

Persona	Goal	Pain Today	What our app delivers
Priya, 24 — Med student	Stick to a fixed study + sleep schedule	Reminders are easy to ignore; books are too slow	Push reminders with 'Mark Done' action + offline sync
Marco, 38 — Founder	Recover from burnout, read 1 book/week	Apps feel sterile, no enjoyable reading flow	Visually rich UI with calm motion; downloadable books
Anya, 17 — High-school athlete	Build a daily training + recovery app	Coaches can't share notes from app	Coaches PDFs and share publicly
Ravi, 45 — Operator / Admin	Run a clean community	No way to scale moderation	Full admin panel with moderation queue and filters

4. Feature Catalogue

Module	Capabilities	Priority
Auth & Onboarding	Email + OAuth (Google, Apple); animated onboarding; timezone capture	P0
Routine Tracker	Create / edit / soft-delete tasks; repeat days; time; categories; drag-reorder	P0
Completion Engine	Mark done / undo; per-day record; optimistic UI; streak counter	P0
Push Reminders	FCM + APNs; per-task schedule; 'Mark Done' action button; quiet hours	P0
Stats & Results	Daily ring, weekly heatmap, streaks, category breakdown, trends	P0
3D / Neumorphic UI	Soft shadows, pressed states, depth tokens, dark/light parity	P0
Scroll-triggered Motion	Section reveals, parallax, sticky headers, hero blur on scroll	P1
E-Book Reader (Online)	Stream PDF / EPUB; pagination; bookmarks; font scaling; night mode	P0
E-Book Reader (Offline)	Download for offline; encrypted local cache; storage manager	P0
User Uploads	Upload PDF/EPUB; cover auto-extract; private by default	P0
Public Library	Toggle book → public; moderation review; categories; search	P1
Admin Panel	Web + RN admin app; user mgmt; moderation; feature flags; broadcasts	P0
Notifications Center	In-app inbox; system push; broadcasts from admin	P1
Settings	Theme, language, quiet hours, data export, account delete (GDPR)	P1
Analytics	Anonymised usage events; retention; reading minutes; admin dashboard	P2

5. Technology Stack

Layer	Technology	Rationale
Mobile App	React Native 0.74 + TypeScript + Expo (bare workflow)	Single codebase for iOS & Android; great dev velocity
Navigation	React Navigation 7 (native-stack + bottom-tabs)	Native gestures, deep links, stable
State	Zustand + React Query (TanStack)	Minimal global store + cached server state with offline support
Animations	Reanimated 3 + Moti + react-native-skia	Framer-Motion-equivalent declarative API; 60–120fps worklets
UI Theming	Custom Neumorphic design tokens + restyle-engine	Type-safe theme, dark/light parity
E-Book Rendering	react-native-pdf + epubjs-rn (or readium-rn)	Page navigation, selection, bookmarks for PDF and EPUB
File Storage	react-native-blob-util + AES-encrypted FS	Offline downloads with at-rest encryption
Backend	Node.js 20 + Fastify (TypeScript)	Lower overhead than Express; first-class TS support
Database	PostgreSQL 16	Relational, reliable for user, task, book and moderation data
ORM	Prisma 5	Type-safe queries; migrations; great DX
Object Storage	S3-compatible (Cloudflare R2 / AWS S3)	Stores uploaded book files and cover images
Auth	Clerk (managed) or self-hosted Lucia + JWT	Email, OAuth, multi-device sessions
RBAC	Custom role + permission service	user / contributor / moderator / admin / super-admin
Push	Firebase Cloud Messaging (Android) + APNs (iOS)	Reliable cross-platform delivery
Scheduler	BullMQ on Redis	Per-user, per-task reminder jobs; retries; cron
Search	Postgres FTS or Meilisearch	Fast title/author search across libraries
Admin Web App	Next.js 14 + shadcn/ui + Tailwind	Same auth + RBAC; reuses backend
DevOps	GitHub Actions; EAS Build; Fastlane	Automated CI, app-store deploys
Monitoring	Sentry (mobile + API) + Grafana Cloud	Crash + perf + business metrics

6. 3D / Neumorphic Design System

Every interactive surface — buttons, cards, toggles, progress rings, task tiles, book covers — uses Neumorphism: soft inner and outer shadows that simulate a pressable surface extruded from the background. We avoid hard borders; depth is communicated entirely through light and shadow.

Design tokens

Token	Light value	Dark value	Purpose
bg.base	#E8ECF2	#1A1D24	Page background
bg.surface	#EDF1F7	#22262F	Card / button surface
shadow.outer.lt	rgba(255,255,255,0.9)	rgba(255,255,255,0.04)	Top-left highlight
shadow.outer.dk	rgba(28,40,75,0.12)	rgba(0,0,0,0.55)	Bottom-right shadow
shadow.inner	inset highlights + 1px shadow	inset darker + lighter rim	Pressed / focused state
radius.md / lg / xl	16 / 22 / 28	16 / 22 / 28	Rounded corners
elevation.1 → 4	y:2 → y:14, blur:6 → 36	scaled darker	Resting / hover / pressed depth
accent.primary	#6D28D9	#A78BFA	Brand violet
accent.success	#16A34A	#4ADE80	Streaks, completion

Button interaction spec

- **Rest:** outer dual-shadow (lt + dk), 6dp elevation.
- **Hover/focus:** elevation 8dp, subtle scale 1.02.
- **Pressed:** inner shadow active, elevation 0, scale 0.98, haptic medium.
- **Disabled:** flatten shadows by 60%, desaturate label.
- All transitions driven by Reanimated withSpring (damping 18, stiffness 220) — never CSS-like easing.

Accessibility

- Provide a 'High Contrast' theme that replaces Neumorphic shadows with WCAG-AA borders.
- Respect prefers-reduced-motion: fall back to instant state changes.
- All buttons expose accessibilityRole='button' with descriptive labels.

7. Motion & Scroll-Triggered Animations

On the web, Framer Motion is the gold standard for declarative motion. In React Native we deliver the same experience with Reanimated 3 (worklets on the UI thread) wrapped by Moti, which exposes a Framer-Motion-style API (`from` / `animate` / `exit` / `transition`).

Scroll-triggered patterns

Surface	Trigger	Effect
Home hero	Vertical scroll 0 → 200dp	Title scales 1 → 0.85, opacity 1 → 0.4, background blur 0 → 12
Section titles	Element enters viewport	Slide-up 24dp + fade-in (spring, 320ms)
Task tiles	List scroll	Stagger 60ms; Neumorphic shadow grows on snap
Streak flame	Tab switch	Shared-element transition; flame morphs across screens
Book cover grid	Pull-to-refresh	Parallax y-translate -20 → 0 with elastic spring
Reader chapter	Page swipe	Skia-rendered 3D page-curl with shadow
Modal sheets	Open / close	Spring scale 0.92 → 1; backdrop opacity 0 → 0.4

Implementation rules

- All scroll listeners use `useAnimatedScrollHandler` — never JS-thread `onScroll`.
- Shared values drive transforms; never set state in scroll callbacks.
- Respect reduced-motion: gate every effect behind `useReducedMotion()`.
- Use `InteractionManager` to defer heavy work until animations finish.

8. E-Book Reader — Online & Offline

The reader supports two source modes (streamed and downloaded) and two file formats (PDF, EPUB). Users can switch between modes per-book.

Online mode

- Book is streamed in ranged chunks from S3-compatible storage via signed URLs (15-minute TTL, re-signed by backend).
- Cover, metadata and current-page state cached locally.
- Reading position synced to backend every 5 seconds (debounced).

Offline mode

- User taps 'Download' on a book detail screen → full file fetched and stored in app's encrypted document directory.
- Files are AES-256-GCM encrypted at rest with a per-user key stored in Keychain (iOS) / EncryptedSharedPreferences (Android).
- A 'Storage Manager' screen lists downloaded books with size and a 'Remove download' action.
- Background sync re-uploads reading position once the device is online.

Reader UI features

- Tap-to-toggle chrome (header + bottom controls).
- Font size +/-, line spacing, three themes (Light / Sepia / Night).
- Bookmarks list with timestamp and progress.
- Highlights and notes (saved per user; visible only to that user).
- Search inside book (FTS for EPUB; rasterised text search for PDF).
- Reading-time tracking → feeds Stats engine.

9. User Uploads & Public Library

Authenticated users can upload PDF or EPUB files up to 50 MB. Each upload is private by default, with an explicit toggle to publish to the Public Library.

Upload pipeline

- Client requests a pre-signed S3 upload URL from `POST /books/upload-url`.
- File is uploaded directly to S3 with content-type validation.
- Server worker scans the file (ClamAV) and extracts metadata (title, author, page count, cover image).
- Book row created with `visibility = 'private'`; owner notified when ready.

Going public

- Owner toggles 'Make Public' on the book → status becomes 'pending_review'.
- Moderator reviews via admin panel: approve, reject (with reason), or request edits.
- Approved books appear in Public Library with author attribution and an option to download to offline storage.
- Owner can revoke public status at any time; book returns to private.
- **Copyright safeguards:** uploader must accept a content-rights checkbox; reporting flow lets any user flag a public book.

Discovery

- Categories (Self-help, Fiction, Study, etc.) + tags.
- Full-text search on title, author, description.
- Sorting: Newest, Trending (7-day downloads), Most read.
- 'Recommended for your routine' surface that maps category → routine tag.

10. Admin Panel & Role Model

Operators manage the platform through a dedicated Next.js admin web app and a slimmed-down RN admin tab. Both share the same backend with fine-grained Role-Based Access Control.

Roles & permissions

Role	Scope
user	Use the app — create routines, upload books, read.
contributor	Upload books and request public publication.
moderator	Review the moderation queue; approve / reject uploads; suspend users; resolve reports.
admin	Everything a moderator can do + manage feature flags, broadcast notifications, edit categories, configure queue.
super_admin	Manage roles for other admins; rotate keys; export full data; delete data on request.

Admin features

- **Dashboard:** DAU/MAU, reading minutes, completion rate, pending moderation count.
- **User management:** search, view profile, suspend, reset password, force-logout.
- **Moderation queue:** preview uploaded books, audit virus scan, approve / reject with templated reasons.
- **Content management:** categories, tags, featured books carousel.
- **Feature flags:** remote toggle for experiments (e.g. 'new reader UI', 'A/B onboarding').
- **Broadcast notifications:** compose and send push to all users or filtered cohorts.
- **Reports:** handle user-flagged content with status workflow.
- **Audit log:** every privileged action is recorded with actor, timestamp, payload diff.

11. Push-Notification Reminder System

Reminders are mission-critical for habit formation. The system is layered for reliability across iOS and Android, online and offline.

Architecture

- User toggles 'Remind me' when creating or editing a task.
- Backend stores task time + repeat days; BullMQ schedules the next occurrence per user in their timezone.
- Worker fires at the scheduled instant → sends via FCM (Android) or APNs (iOS).
- Notification payload contains an 'action' button: **Mark Done**, which calls `POST /completions` without opening the app.
- If the app is open, an in-app Neumorphic banner overlay is shown instead of a system notification.
- **Local fallback:** the device also schedules a local notification via Notifee so reminders fire even if the device is offline.

Quiet hours & preferences

- User-configurable quiet hours (e.g. 22:00 → 07:00) — silent push only.
- Per-category mute (e.g. mute 'work' on weekends).
- Snooze 5 / 15 / 60 min directly from the notification.

Admin broadcasts

- Admins can compose and send broadcast pushes to all users or filtered cohorts (timezone, role, last-active).
- Broadcasts are rate-limited and require `super_admin` approval for audiences over 10k.

12. Database Schema

All user data is isolated by userId on every query. Books, completions, and notifications are soft-deletable for undo and audit. The schema below extends v1.0.

User

id UUID PK • email VARCHAR(255) UNIQUE • name VARCHAR(100) • timezone VARCHAR(50) • role ENUM(user, contributor, moderator, admin, super_admin) DEFAULT 'user' • avatarUrl TEXT • locale VARCHAR(10) • createdAt • lastActiveAt • suspendedAt NULL

Task

id UUID PK • userId FK • title VARCHAR(200) • description TEXT • category ENUM(morning, work, health, evening, study, reading, other) • time TIME • repeatDays TEXT[] • remindEnabled BOOLEAN • quietHoursOverride BOOLEAN • linkedBookId UUID FK NULL -- optional: pair routine with a book • createdAt • deletedAt NULL

Completion

id UUID PK • taskId FK • userId FK • completedAt DATE • skipped BOOLEAN

Streak

id UUID PK • userId FK UNIQUE • currentStreak INT • longestStreak INT • lastCompletedDate DATE

Book

id UUID PK • ownerId FK • title • author • description • fileKey TEXT -- S3 object key • coverKey TEXT • format ENUM(pdf, epub) • sizeBytes BIGINT • pageCount INT • visibility ENUM(private, pending_review, public, rejected) DEFAULT 'private' • reviewedBy FK NULL • reviewedAt NULL • rejectionReason TEXT NULL • categoryId FK NULL • tags TEXT[] • downloadsCount INT DEFAULT 0 • createdAt • deletedAt NULL

BookCategory

id UUID PK • name • slug • icon • orderIndex

ReadingProgress

id UUID PK • userId FK • bookId FK • currentLocator TEXT -- CFI or page • percent NUMERIC(5,2) • secondsRead INT • lastReadAt

BookDownload

id UUID PK • userId FK • bookId FK • downloadedAt • localSizeBytes • status

Bookmark / Highlight

id UUID PK • userId FK • bookId FK • locator TEXT • note TEXT • color • createdAt

Report

id UUID PK • reporterId FK • bookId FK • reason ENUM(copyright, abuse, spam, inaccurate, other) • detail TEXT • status ENUM(open, resolved, dismissed) • resolvedBy FK NULL • resolvedAt NULL

NotificationLog

id UUID PK • userId FK • type ENUM(reminder, broadcast, system) • payload JSONB • sentAt •

deliveredAt NULL • openedAt NULL

FeatureFlag

id UUID PK • key VARCHAR(80) UNIQUE • enabled BOOLEAN • rolloutPercent INT • audience JSONB

AuditLog

id UUID PK • actorId FK • action TEXT • targetType • targetId UUID • diff JSONB • ip • userAgent • createdAt

13. REST + Realtime API Endpoints

Versioned under `/api/v1`. All routes except `/auth/*` require a valid Bearer JWT. Realtime events (book status, broadcasts) use WebSockets at `/ws`.

Method	Endpoint	Description
POST	<code>/auth/register</code>	Create user (email + password / OAuth)
POST	<code>/auth/login</code>	Authenticate; returns JWT + refresh
POST	<code>/auth/refresh</code>	Rotate refresh token
POST	<code>/auth/logout</code>	Invalidate refresh
GET	<code>/auth/me</code>	Current user profile + role
GET / POST	<code>/tasks</code>	List / create routine tasks
PATCH / DELETE	<code>/tasks/:id</code>	Update / soft-delete a task
GET / POST	<code>/completions</code>	Mark / list completions
DELETE	<code>/completions/:id</code>	Undo completion
GET	<code>/stats/weekly /streaks /categories /reading</code>	Aggregated stats per user
POST	<code>/books/upload-url</code>	Get a pre-signed S3 upload URL
POST	<code>/books</code>	Register a book after upload (metadata)
GET	<code>/books?scope=mine public&search=</code>	List books with filters
GET	<code>/books/:id</code>	Book detail incl. signed read URL
PATCH	<code>/books/:id</code>	Update metadata / visibility
DELETE	<code>/books/:id</code>	Soft-delete (owner or admin)
POST	<code>/books/:id/download</code>	Issue download manifest + signed URL
POST	<code>/books/:id/report</code>	Flag a public book
GET / PUT	<code>/reading-progress/:bookId</code>	Sync reading position
GET / POST / DELETE	<code>/bookmarks</code>	Bookmarks + highlights
GET	<code>/admin/moderation?status=pending</code>	Moderation queue
POST	<code>/admin/moderation/:id/approve /reject</code>	Moderation actions
GET / PATCH	<code>/admin/users/:id</code>	Inspect / update user (role, suspend)
GET / POST	<code>/admin/feature-flags</code>	List / upsert flags
POST	<code>/admin/broadcasts</code>	Send push to an audience
GET	<code>/admin/audit-log</code>	Privileged action history
POST	<code>/notifications/devices</code>	Register FCM / APNs token
GET	<code>/notifications/inbox</code>	In-app notifications

Method	Endpoint	Description
GET	/account/export	GDPR data export (signed URL)
DELETE	/account	Full account deletion (GDPR)

14. Frontend Architecture (React Native)

Folder structure

```
app/  
  ■■■■ (auth)/ Sign-in, sign-up, reset, onboarding  
  ■■■■ (tabs)/  
  ■ ■■■■ today.tsx Routine dashboard (today)  
  ■ ■■■■ library.tsx Personal + public books  
  ■ ■■■■ reader.tsx E-book reader (modal-stack)  
  ■ ■■■■ stats.tsx Streaks, charts, reading minutes  
  ■ ■■■■ settings.tsx Profile, theme, quiet hours  
  ■■■■ (admin)/ Admin tabs (visible only to roles ≥ moderator)  
src/  
  ■■■■ components/  
  ■ ■■■■ neu/ NeuButton, NeuCard, NeuToggle, NeuRing, NeuSlider  
  ■ ■■■■ motion/ FadeInOnScroll, ParallaxImage, StaggerList  
  ■ ■■■■ tasks/ TaskCard, TaskForm, CategoryTag  
  ■ ■■■■ reader/ Pager, Toolbar, ProgressBar, FontSheet  
  ■ ■■■■ admin/ ModerationRow, FlagEditor, BroadcastForm  
  ■■■■ theme/ tokens.ts, neumorphic.ts, motion.ts  
  ■■■■ hooks/ useReminders, useBooks, useReader, useRoles  
  ■■■■ lib/ api client (axios), encryption, file storage  
  ■■■■ store/ Zustand slices (auth, tasks, reader, admin)  
  ■■■■ types/ Task, Book, User, Notification, Flag interfaces
```

Key design decisions

- **Expo Router** for typed file-based routing.
- **Zustand** for client state; **React Query** for server state with offline-first cache.
- **MMKV** as the persistence layer (faster than AsyncStorage).
- Reanimated worklets are the only place UI-thread animations live.
- All Neumorphic primitives are wrapped — engineers never style shadows by hand.
- Role-gated routes via `<RequireRole>`.

15. Backend Architecture

- Fastify app split into modules: auth, tasks, books, admin, notifications, stats.
- Each module exposes routes + a service + Prisma queries; no cross-module imports of internals.
- **Workers** (separate processes): reminder scheduler, upload post-processor, virus scanner, broadcast sender.
- **Redis** hosts BullMQ queues + caches stats + holds rate-limit counters.
- **S3 / R2** stores book files and covers; access via signed URLs only — no public buckets.
- **Webhook receivers** for Stripe (future monetisation) and Sentry alerts.

16. Security, Auth & Privacy

Concern	Implementation
Authentication	JWT access (15min) + rotating refresh (7d); device-bound sessions
Authorization	RBAC middleware; every query scoped by userId or role
Input validation	Zod schemas on every route body, params and query
Rate limiting	100 req / 15min / IP global; stricter on /auth/* and upload
Data isolation	All multi-tenant queries filter by userId or visibility rules
File scanning	ClamAV scan of every uploaded file before activation
At-rest encryption (device)	AES-256-GCM on downloaded books; key in Keychain / Encrypted Prefs
At-rest encryption (cloud)	S3 SSE-KMS for all book objects
Transport	TLS 1.2+; HSTS; certificate pinning in mobile app
Secrets	Managed via Doppler / AWS Secrets Manager; never in source
GDPR / DSAR	/account/export + /account DELETE with full cascade
Audit	AuditLog row for every admin action; immutable retention 1y

17. Stats & Results Engine

Metric	How calculated	Surfaced as
Daily progress	$\text{done} \div \text{total today} \times 100\%$	Neumorphic circular ring
Current streak	Consecutive 100% days	Flame badge with count
Best streak	Max in Streak table	Trophy badge
Category score	$\text{Done} \div \text{total per category} / \text{week}$	Horizontal Neu-bars
Weekly heatmap	Completion % per day (7d)	7-cell color grid
Reading minutes	$\text{Sum}(\text{ReadingProgress.secondsRead}) / \text{week}$	Bar chart
Books finished	$\text{Count}(\text{percent} \geq 100\%) \text{ ever}$	Trophy carousel
Consistency rate	% scheduled days done (30d)	Percent + trend arrow

18. DevOps & Deployment Pipeline

#	Stage	Actions
1	Code push	Feature branch → PR; conventional commits enforced
2	Static checks	ESLint, Prettier, TypeScript, detox-lint on every PR
3	Tests	Vitest (unit), RTL (component), Supertest (API), Maestro (E2E)
4	Backend build & deploy	Docker image → Railway / Fly.io staging on merge to main
5	Mobile build	EAS Build produces signed iOS and Android binaries
6	QA channels	TestFlight (iOS) + Internal track (Play Store)
7	Production release	Manual approval → store submission via Fastlane
8	Post-deploy	Sentry release tracking; Datadog dashboards; auto-rollback on error spikes

19. Development Phases & Timeline

Estimated 12 weeks from kickoff to soft launch — longer than v1 because of the e-book reader, uploads, admin panel and native build pipeline.

Phase	Weeks	Scope
1 — Foundations	1–2	Auth, RBAC, base navigation, Neumorphic design system tokens
2 — Routines	3–4	Task CRUD, completions, streak engine, optimistic UI
3 — Reminders	5	FCM + APNs, BullMQ scheduler, local fallback, quiet hours
4 — Stats & Motion	6	Stats engine, scroll-triggered animations, shared-element transitions
5 — E-Book Reader	7–8	PDF + EPUB rendering, online stream, offline encrypted download
6 — Uploads & Library	9	Upload pipeline, public library, search, reports
7 — Admin Panel	10	Next.js dashboard, moderation, feature flags, broadcasts
8 — Polish & Launch	11–12	Accessibility, performance, QA, store submission, soft launch

20. Non-Functional Requirements

Requirement	Target	How achieved
App cold-start	< 2.5s on mid-range Android	Hermes engine, asset preloading, code splitting
Animation perf	60fps p95	Reanimated worklets, no JS-thread animations
API latency	p95 < 250ms	Indexed Postgres queries; Redis cache; HTTP/2
Offline support	Read + complete tasks fully offline	MMKV cache + queued writes synced on reconnect
Accessibility	WCAG 2.1 AA	Labels, focus order, high-contrast theme, reduced motion
Scalability	100k MAU	Stateless API + horizontal scaling + connection pooling
Reliability	99.9% uptime	Multi-AZ DB, health checks, auto-restart
Privacy	GDPR + CCPA	Export + delete endpoints; minimal data collection
Crash-free sessions	> 99.5%	Sentry; smoke tests before release

21. Testing Strategy

Type	Tool	Coverage	What's tested
Unit	Vitest	>80%	Business logic, utilities, Zustand stores
Component	RTL + jest-native	>70%	UI components, interactions, a11y
API	Supertest + Jest	100% endpoints	All routes, auth, error cases, RBAC
E2E mobile	Maestro / Detox	Critical flows	Sign up → add task → complete; upload → publish
E2E admin	Playwright	Critical flows	Moderation, broadcasts, flags
Perf	Reassure + Lighthouse CI	Score >90	Re-renders; admin web load
Load	k6	1k concurrent	Auth, list books, completion writes
Security	OWASP ZAP + npm audit	All releases	Auth, RBAC bypass, injection

22. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Copyrighted uploads in public library	Medium	High	Mandatory rights checkbox, ClamAV + content review, takedown
Notification deliverability gaps on iOS	Medium	High	APNs + Notifee local fallback; per-platform monitoring
EPUB/PDF rendering edge cases	Medium	Medium	Use battle-tested libraries; reader smoke-test corpus per release
Storage cost from public library	Low	Medium	Per-user upload quota; cold-storage older books; rate-limit downl
Admin abuse / privilege escalation	Low	High	Immutable audit log; 2FA mandatory for admin+ roles; least-priv
Performance regression with heavy mod	Medium	Medium	Reanimated only; per-PR Reassure perf budget

23. Master AI Build Prompt

Paste the prompt below into your AI coding agent. It is self-contained and references every requirement above so the agent can scaffold and implement the project end-to-end.

ROLE

You are a senior full-stack engineer. Build a production-ready cross-platform mobile application called **"Daily Routine & E-Book Tracker"**. Deliver working source code, tests, infrastructure-as-code and store-submission artefacts. Pause and ask the user only when a decision is genuinely ambiguous; otherwise pick the best default consistent with this spec and proceed.

STACK (use exactly these unless impossible)

- Mobile: React Native 0.74 + TypeScript via Expo (custom dev client), Expo Router, React Navigation 7.
- Animations: Reanimated 3 + Moti + react-native-skia (Framer-Motion-style API).
- State: Zustand + React Query (TanStack); MMKV for persistence.
- Backend: Node.js 20 + Fastify + TypeScript; Prisma 5 on PostgreSQL 16; Redis + BullMQ; S3-compatible object storage (Cloudflare R2 or AWS S3).
- Auth: Clerk (default) with RBAC roles user / contributor / moderator / admin / super_admin.
- Push: Firebase Cloud Messaging (Android) + APNs (iOS), with Notifee local fallback.
- E-Book rendering: react-native-pdf for PDF, epubjs-rn (or readium-mobile) for EPUB. AES-256-GCM encryption for offline files, key in Keychain / EncryptedSharedPreferences.
- Admin web: Next.js 14 + Tailwind + shadcn/ui sharing the same backend.
- Monitoring: Sentry (mobile + API) and Grafana Cloud.
- CI/CD: GitHub Actions; EAS Build; Fastlane for store submission.

VISUAL LANGUAGE

Implement a strict **Neumorphic 3D UI**: every button, card, toggle, progress ring and book cover uses dual outer shadows (light top-left + dark bottom-right) with rounded corners 16–28. Pressed state uses inner shadows, scale 0.98 and haptic medium. Provide light and dark themes plus a "High Contrast" theme that swaps shadows for WCAG-AA borders. Honor prefers-reduced-motion at every animation site.

MOTION

Use Reanimated worklets and Moti exclusively (no Animated API, no JS-thread animations). Implement scroll-triggered patterns:

- Hero scales/fades/blurs based on scrollY.
- Section titles slide-up + fade-in on enter-viewport.
- Task tiles stagger 60ms.
- Shared-element transitions for streak flame and book covers between screens.
- Skia-based 3D page-curl in the reader.

FEATURES (all P0 unless noted)

1. **Auth & Onboarding** — email + Google + Apple; capture timezone; animated 4-screen onboarding.
2. **Routine Tracker** — CRUD tasks with title, category, time, repeat days, linked book; optimistic UI; soft delete.
3. **Completion Engine** — mark done / undo; one record per task per day; update streak.
4. **Reminders** — per-task scheduling in user's timezone via BullMQ; push with "Mark Done" action;

in-app banner when foreground; Notifee local fallback offline; quiet hours; snooze 5/15/60.

5. **Stats** — daily ring, streaks, weekly heatmap, category breakdown, reading minutes.

6. **E-Book Reader** — online streaming via signed URLs and offline download with AES-256-GCM encryption; bookmarks; highlights; font sizing; Light / Sepia / Night themes; reading-progress sync.

7. **User Uploads** — upload PDF/EPUB ≤50MB via pre-signed URL; ClamAV scan; auto-extract cover and metadata; private by default.

8. **Public Library** — owner toggles public → moderation queue → approve/reject; categories, tags, search, sorting; report flow; "Recommended for your routine".

9. **Admin Panel** — Next.js web app + RN admin tab. Dashboard, user mgmt (suspend, reset, role change), moderation queue, feature flags with rollout %, broadcast push composer with cohort filters, reports workflow, immutable audit log. Gate by role.

10. **Settings** — theme, language, quiet hours, notification prefs, GDPR export & delete.

DATA MODEL

Implement Prisma schema covering User, Task, Completion, Streak, Book, BookCategory, ReadingProgress, BookDownload, Bookmark/Highlight, Report, NotificationLog, FeatureFlag, AuditLog. All user-scoped queries must filter by `userId`. Books have visibility ∈ {private, pending_review, public, rejected}.

API

Versioned REST under `/api/v1` with JWT bearer auth, plus WebSocket `/ws` for realtime moderation + broadcast events. Implement every endpoint listed in section 13 of the PRD. Use Zod schemas for validation, rate-limit `/auth/*` and uploads, and return RFC 7807 problem-detail errors.

SECURITY

RBAC middleware; 2FA mandatory for moderator+; signed URLs with 15-minute TTL; certificate pinning on mobile; AuditLog row on every privileged action; OWASP ZAP scan in CI; npm audit gating release.

QUALITY BAR

- TypeScript strict everywhere; no any.
- ESLint + Prettier; conventional commits; PRs require green CI + 1 review.
- Tests: Vitest unit >80%, RTL component >70%, Supertest API 100% of routes, Maestro E2E for sign-up → add task → complete and upload → publish flows, Playwright E2E for admin moderation.
- Performance: app cold-start <2.5s on mid-range Android; animations 60fps p95; API p95 <250ms.
- Accessibility: WCAG 2.1 AA; every interactive element has a label; respect reduced motion.

DELIVERABLES

1. Monorepo (pnpm workspaces): apps/mobile, apps/admin-web, apps/api, packages/ui, packages/types, packages/config.
2. README with setup, env vars, scripts, architecture diagram.
3. Prisma migrations and seed script with demo data (1 admin, 2 users, 5 books, sample completions).
4. GitHub Actions workflows: lint, test, build, EAS preview, deploy.
5. Fastlane lanes for TestFlight and Play internal track.
6. Docker Compose for local Postgres + Redis + Minio.
7. Storybook (mobile + web) for the Neumorphic component library.
8. End-to-end demo video script (Maestro flow recording).

EXECUTION PLAN

Follow the 12-week phase plan in section 19 of the PRD. After each phase print a short progress summary (what was built, what's next, blockers). For any decision not covered above, default to the simplest production-grade option and note it in the README's "Decisions" log.

OUTPUT FORMAT

Stream files using clearly delimited code blocks with full paths. Do not elide code. When introducing a new module, also generate its tests. Finish with a "How to run" section and a checklist mapped to the PRD.

End of document. This PRD plus the master prompt above is sufficient to drive a coding agent (or an engineering team) from zero to a production-ready release.